

YouTube RAG Chatbot

A Zero-Cost, Fully Local Retrieval-Augmented Generation System

Project Documentation & Technical Report

Prepared as a portfolio project for AI/ML and Generative AI engineering roles. Covers problem statement, architecture, technology choices, implementation pipeline, engineering challenges, and interview preparation notes.

Table of Contents

1. Executive Summary
2. Problem Statement — What This Project Solves
3. Project Goals & Scope
4. Why Zero-Cost / Fully Local?
5. System Architecture
6. Technology Stack & Justification
7. Stage-by-Stage Pipeline Explanation
8. Key Concepts Explained (RAG, Embeddings, Chunking, FAISS, LLMs)
9. Engineering Challenges Faced & How They Were Solved
10. Current Limitations
11. Future Improvements Roadmap
12. Interview Preparation Notes
13. Conclusion

1. Executive Summary

This project is a **YouTube RAG (Retrieval-Augmented Generation) Chatbot** — a web application that allows a user to paste any YouTube video URL and then ask natural-language questions about the video's spoken content. The system fetches the video's transcript, processes and semantically indexes it, and uses a locally-run open-source Large Language Model (LLM) to generate accurate, grounded answers with supporting timestamps — without relying on any paid API, cloud LLM service, or paid vector database.

The defining constraint of this build is **zero ongoing API cost**: every component — embeddings, vector search, and language generation — runs entirely on the developer's own machine using open-source tools (Sentence Transformers, FAISS, and Ollama). This makes the project fully reproducible by anyone, with no billing risk, while still demonstrating a complete, production-shaped RAG architecture.

At a glance

Aspect	Detail
Project type	Retrieval-Augmented Generation (RAG) chatbot
Data source	Any public YouTube video (via URL)
Frontend	Streamlit (Python web UI framework)
Embedding model	Sentence Transformers — all-MiniLM-L6-v2 (local)
Vector database	FAISS (Facebook AI Similarity Search) — local, in-memory
LLM	Llama 3.2 (3B) served locally via Ollama
Cost to run	\$0 — no paid APIs, no cloud services
Hardware target	Standard laptop — 8GB RAM, CPU-only, no dedicated GPU required

2. Problem Statement — What This Project Solves

YouTube videos — podcasts, interviews, lectures, tutorials — often contain valuable information buried inside long-form spoken content (30 minutes to several hours). Finding a specific piece of information inside such a video today typically requires:

- Watching or scrubbing through the entire video manually, or
- Reading through a raw, unstructured transcript wall of text (if even available), or
- Relying on generic web search, which has no awareness of what was actually said in that specific video.

None of these are efficient for a viewer who has a *specific question* — e.g. "What did the guest say about X?" or "What supplements did they recommend?". There is no simple way to directly **ask a question and get an answer sourced from that exact video**.

The core problem, restated simply

How do we let a user ask a natural-language question about a YouTube video's content, and receive an accurate answer — grounded in what was actually said, with the ability to trace the answer back to the exact moment in the video — without needing to watch or read the whole thing?

Why a plain LLM chatbot alone can't solve this

A general-purpose LLM (like a base ChatGPT-style model with no added context) either (a) has never seen the specific video's transcript at all, since it wasn't part of its training data, or (b) if it has vague familiarity with the video/creator, it will rely on generic, possibly outdated, or simply invented knowledge — a phenomenon known as **hallucination**. A chatbot answering purely from its own memory cannot reliably quote or reference a specific video's actual spoken content.

Why RAG is the right solution

Retrieval-Augmented Generation (RAG) solves this by combining two steps at answer-time: first, *retrieve* the most relevant snippets of the actual transcript for the user's specific question using semantic search; then, *feed those snippets directly into the LLM's prompt* as grounding context before it generates a response. The model is explicitly instructed to answer only from that retrieved context, and to say so clearly when the answer isn't present — directly addressing the hallucination problem rather than hoping the model "just knows."

3. Project Goals & Scope

Functional goals

- Accept any valid YouTube URL and extract the video ID robustly across URL formats.
- Fetch video metadata (title, channel, thumbnail) for user-facing confirmation.
- Extract the full available transcript/captions for the video.
- Clean and preprocess noisy transcript text (e.g. remove [Music]/[Applause] tags, normalize whitespace).
- Split the transcript into semantically meaningful, appropriately-sized chunks with overlap.
- Generate vector embeddings for each chunk using a local embedding model.
- Store and index those embeddings for fast similarity search.
- Accept free-form user questions through a chat-style interface.
- Retrieve the most relevant transcript chunks for each question via semantic search.
- Generate a grounded, hallucination-resistant answer using a locally-run open-source LLM.
- Display the answer alongside its supporting source chunks and video timestamps.

Non-functional goals

- **Zero recurring cost** — no paid APIs, no paid hosting, no paid vector database.
- **Runs on modest consumer hardware** — designed and tuned for an 8GB RAM, CPU-only laptop.
- **Modular, readable codebase** — each pipeline stage isolated into its own file under `src/`, so the system is easy to explain, extend, and debug.
- **Explainability** — every design decision (model choice, chunk size, index type) is justified against concrete alternatives, not chosen arbitrarily.

Out of scope (for the current version)

- Multi-video comparison / cross-video question answering.
- Production-grade authentication, multi-user support, or persistent cloud storage.
- Non-English transcript optimization (current embedding model is English-optimized; see Limitations).
- Public cloud deployment with the local LLM included (Ollama requires a local machine to run).

4. Why Zero-Cost / Fully Local?

A deliberate constraint of this project was to avoid all paid APIs — no OpenAI, no Gemini, no Claude API, no paid vector database, no paid hosting. This was not simply a budget limitation; it was treated as a **design constraint** that shaped every architectural decision, and it doubles as a strong engineering narrative for interviews: building a genuinely functional AI system without dependency on a billed, externally-controlled service.

Cost comparison: paid API approach vs. this project's approach

Component	Typical Paid Approach	This Project's Approach
Embeddings	OpenAI text-embedding-3 (paid per token)	Sentence Transformers, local, free, unlimited use
Vector DB	Pinecone / Weaviate Cloud (paid tiers)	FAISS, local, in-memory, free
LLM	GPT-4 / Claude / Gemini API (paid per token)	Llama 3.2 (3B) via Ollama, local, free
Hosting	Cloud server with GPU (paid)	Developer's own laptop, CPU-only

This approach also removes a common failure mode of tutorial-style AI projects: an API key expiring, a free credit tier running out, or a demo silently breaking during an interview because a billed key stopped working. Everything here runs entirely offline once models are downloaded, which makes the project fully self-contained and reliable to demo at any time.

5. System Architecture

The system follows a linear RAG pipeline, split into an **indexing phase** (runs once per video) and a **query phase** (runs once per user question):

High-level architecture diagram

YouTube URL → Video ID Extraction → Metadata Fetch (oEmbed) → Transcript Fetch (youtube-transcript-api) → Cleaning & Preprocessing → Chunking (RecursiveCharacterTextSplitter) → Embedding Generation (Sentence Transformers) → FAISS Vector Index

User Question → Query Embedding → Semantic Search (FAISS) → Top-K Relevant Chunks → RAG Prompt Construction (context + question + rules) → Local LLM (Ollama / Llama 3.2) → Grounded Answer + Source Timestamps

Component responsibilities

Module	Responsibility
src/youtube.py	URL validation, video ID extraction (regex), metadata fetch via YouTube oEmbed (no API key)
src/transcript.py	Fetches raw transcript/captions via youtube-transcript-api, handles missing/disabled captions gracefully
src/text_processing.py	Cleans raw segments, builds a character-to-timestamp offset map, chunks text with overlap
src/embeddings.py	Loads and caches the Sentence Transformer model; embeds chunks and user queries into the same vector space
src/vector_store.py	Wraps a FAISS index plus parallel chunk metadata; exposes add(), search(), and a high-level retrieve_relevant_chunks()
src/llm.py	Builds the grounded RAG prompt (system rules + context + question) and calls the local Ollama LLM
app.py	Streamlit UI: URL input, video card, chat interface, session-state management, orchestrates all modules above

Why this modular structure?

Each pipeline stage is isolated into its own file with a single clear responsibility, following the **separation of concerns** principle. This means: (1) any stage can be tested independently (as was done throughout development via standalone test scripts before wiring into the UI), (2) any component can be swapped later — e.g. replacing FAISS with a different vector database, or swapping the LLM — without touching unrelated code, and (3) the codebase reads clearly enough to explain confidently in a technical interview, stage by stage.

6. Technology Stack & Justification

Every core technology choice below was made by weighing it against realistic alternatives — not by default habit.

6.1 Frontend — Streamlit

Option	Trade-off
Streamlit	Pure Python, no HTML/CSS/JS required, extremely fast to prototype a data/AI app UI
Flask/Django + custom frontend	Far more control, but requires separate frontend code (HTML/CSS/JS) and much more setup time
Gradio	Also simple and Python-based, but Streamlit offers more layout/styling flexibility for a multi-section app like this one

Chosen: Streamlit. Streamlit lets the project focus engineering effort on the RAG pipeline itself rather than frontend plumbing, while still allowing full custom CSS styling for a polished, non-generic look.

6.2 Transcript Extraction — youtube-transcript-api

Option	Trade-off
youtube-transcript-api	Free, no API key, directly fetches YouTube's own caption tracks
YouTube Data API v3 (captions endpoint)	Requires an API key and OAuth for caption downloads in most cases; more setup for less benefit here
Web scraping the video page	Fragile, breaks on YouTube UI/DOM changes, and against the spirit of using stable public interfaces

Chosen: youtube-transcript-api (open-source Python library). This library talks to the same caption-serving mechanism YouTube's own player uses, requires zero authentication, and returns clean, timestamped segments out of the box.

6.3 Metadata — YouTube oEmbed endpoint

Option	Trade-off
oEmbed endpoint	No API key at all; returns title, author/channel, enough for the UI card
YouTube Data API v3	Richer metadata (views, duration, description) but requires a Google Cloud project and API key

Chosen: YouTube's public oEmbed endpoint. Since the project's priority is zero setup friction, oEmbed was chosen for MVP metadata. The Data API is flagged as an optional future upgrade for richer metadata, clearly documented as needing a free-tier key.

6.4 Chunking — LangChain's RecursiveCharacterTextSplitter

Option	Trade-off
RecursiveCharacterTextSplitter	Tries paragraph/sentence/word boundaries in order before falling back to raw character cuts
Fixed-size character slicing (custom)	Simple to write, but frequently cuts mid-sentence, hurting chunk quality
Sentence-only splitting	Ignores desired chunk-size control; sentence lengths vary too much for consistent embeddings

Chosen: RecursiveCharacterTextSplitter (from langchain-text-splitters). This splitter balances chunk-size consistency with respecting natural language boundaries, and its chunk_overlap parameter directly supports overlap-based context preservation, a standard RAG technique.

6.5 Embeddings — Sentence Transformers (all-MiniLM-L6-v2)

Option	Trade-off
all-MiniLM-L6-v2 (~80MB, 384-dim)	Fast on CPU, small footprint, strong quality-to-size ratio; the de facto standard 'good default' embedding model
all-mpnet-base-v2 (~420MB, 768-dim)	Slightly higher quality but ~5x heavier — noticeably slower on CPU-only, 8GB RAM hardware
OpenAI text-embedding-3	Best-in-class quality, but paid per token and requires an internet connection and API key — violates the zero-cost constraint

Chosen: all-MiniLM-L6-v2 via the sentence-transformers library. Given the target hardware (8GB RAM, no dedicated GPU), all-MiniLM-L6-v2 offers the best practical balance: fast enough to embed hundreds of chunks in seconds, and free/local, with quality sufficient for this scale.

6.6 Vector Database — FAISS

Option	Trade-off
FAISS IndexFlatIP	Exact (brute-force) cosine-similarity search; free, local, extremely fast at this project's scale (hundreds of vectors per video)
Pinecone / Weaviate Cloud	Managed, scalable to millions of vectors and multi-user access, but paid and adds network latency + infra complexity not needed here
FAISS IndexIVFFlat / HNSW	Approximate search, faster at massive scale (millions of vectors) — unnecessary overhead for a single-video, few-hundred-chunk use case

Chosen: FAISS (IndexFlatIP), local and in-memory. At the scale of one video's transcript (typically tens to a few hundred chunks), an exact brute-force FAISS index is both simpler and effectively instant — no need for the added complexity of approximate indexing or a hosted service.

6.7 LLM Serving — Ollama (Llama 3.2, 3B)

Option	Trade-off
Ollama + Llama 3.2 (3B)	Free, fully local, no API key; runs comfortably within an 8GB RAM CPU-only budget; simple HTTP/Python API
GPT-4 / Claude / Gemini APIs	Higher quality, but paid per token — directly violates the zero-cost requirement
Larger local models (7B+, e.g. Mistral, Llama 3.1 8B)	Better quality, but risk heavy RAM swapping/freezing on an 8GB-RAM CPU-only machine

Chosen: Ollama running Llama 3.2 (3B, quantized). Ollama abstracts away model quantization, downloading, and serving behind a clean local API (built on llama.cpp under the hood), and the 3B model size was chosen specifically to fit the target hardware profile without sacrificing usable answer quality.

7. Stage-by-Stage Pipeline Explanation

Stage 1 — URL Validation & Video ID Extraction

The user-provided URL is matched against several regex patterns covering YouTube's different URL shapes (standard watch links,youtu.be short links, embed links, shorts links) to reliably extract the 11-character video ID regardless of format.

Stage 2 — Metadata Fetch

The extracted video ID is used to call YouTube's public oEmbed endpoint, returning the video's title, channel name, and a constructed thumbnail URL — all without requiring an API key.

Stage 3 — Transcript Extraction

youtube-transcript-api fetches the full timestamped transcript for the video. Each transcript segment retains its original start time and duration, which is essential context preserved for later timestamp citation. Videos with disabled or missing captions are handled with a clear, non-crashing error message.

Stage 4 — Cleaning & Preprocessing

Each segment's text is cleaned: bracketed noise annotations (e.g. [Music], [Applause]) are stripped via regex, and irregular whitespace/line breaks are collapsed. Segments that become empty after cleaning are dropped. An 'offset map' is also built here, recording which character range of the full concatenated transcript text corresponds to which original timestamp — this is what allows timestamp information to survive the upcoming chunking step.

Stage 5 — Chunking

The cleaned, concatenated transcript text is split into overlapping chunks (default: ~800 characters, ~150-character overlap) using LangChain's RecursiveCharacterTextSplitter. Each resulting chunk is then matched back to its approximate start timestamp using the offset map from Stage 4.

Stage 6 — Embedding Generation

Each chunk's text is converted into a 384-dimensional normalized vector using the all-MiniLM-L6-v2 Sentence Transformer model. Normalization (unit length) is applied so that cosine similarity search reduces to a fast inner-product computation later.

Stage 7 — Vector Indexing

All chunk embeddings are added to a FAISS IndexFlatIP index. A parallel Python list stores each chunk's original text and timestamp, kept in the same order as vectors in the index, so a FAISS match can always be traced back to its source text.

Stage 8 — Query-Time Retrieval

When the user asks a question, it is embedded using the exact same model and normalization used for the chunks (critical — query and chunk embeddings must share the same vector space to be comparable). The FAISS index is searched for the top-K (default 4) most similar chunks by cosine similarity.

Stage 9 — RAG Prompt Construction

The retrieved chunks are formatted into a labeled context block (each tagged with its timestamp), combined with the user's question, and paired with a system prompt instructing the model to answer only from the provided context and to explicitly decline when the answer is not present.

Stage 10 — Answer Generation

The full prompt (system instructions + context + question) is sent to the locally-running Ollama server via its Python client, which returns a generated, grounded answer.

Stage 11 — Answer & Source Display

The generated answer is shown prominently in the chat interface, with the specific source chunks and their video timestamps available in a collapsible section beneath it — giving the user both the answer and a way to verify/trace it back to the original content.

8. Key Concepts Explained

What is RAG (Retrieval-Augmented Generation)?

RAG is a technique that combines a search/retrieval step with a language model's generation step. Instead of relying solely on what an LLM memorized during training, RAG retrieves relevant external information at the moment of answering and feeds it into the model's prompt as grounding context. This makes answers more accurate, current, and traceable to a real source.

What are embeddings?

An embedding is a fixed-length numeric vector representing the semantic meaning of a piece of text. Texts with similar meaning are positioned close together in this vector space, even if they share few or no exact words — which is what enables meaning-based (semantic) search rather than simple keyword matching.

What is semantic search?

Semantic search finds content based on meaning rather than exact keyword overlap. A question and its answer can be phrased completely differently in words but still be recognized as related, because their embeddings land close together in vector space.

What is chunking, and why use chunk overlap?

Chunking splits long text into smaller, more manageable pieces suited to embedding and retrieval — small enough to be precise, large enough to contain a complete idea. Overlap between consecutive chunks ensures that an idea or fact spanning a chunk boundary still appears intact within at least one chunk, preventing it from being split awkwardly and becoming unretrievable.

Why use FAISS?

FAISS provides a fast, free, local way to search through a collection of vectors for the most similar matches to a query vector. At the scale of a single video's transcript (hundreds of chunks), FAISS's exact search is effectively instantaneous, with zero infrastructure or cost overhead.

What does a retriever do?

A retriever is the component responsible for converting a user's question into an embedding and searching the vector index for the most relevant stored chunks — effectively 'finding the right paragraphs' before the LLM ever sees the question.

How does the LLM receive retrieved context?

The retrieved chunks are inserted directly into the text of the prompt sent to the LLM, alongside explicit instructions on how to use them (answer only from this context; say so if the answer isn't present). The LLM does not query the vector store itself — it only ever sees plain text handed to it in the prompt.

Why can RAG reduce hallucination?

By grounding generation in retrieved, relevant source text and instructing the model to decline when the context doesn't contain an answer, RAG reduces (though does not fully eliminate) the model's tendency to invent plausible-sounding but false information, since it has real material to draw from instead of

relying purely on parametric memory.

Why use an open-source local LLM instead of a paid API?

A local LLM via Ollama removes per-token cost, works offline, and avoids dependency on a third-party service's uptime, pricing changes, or key management — at the cost of somewhat lower output quality compared to top-tier proprietary models, which is an acceptable trade-off for this project's goals.

What is Ollama?

Ollama is an open-source tool that downloads, manages, and serves quantized open-source LLMs locally, exposing them through a simple local API (default: <http://localhost:11434>). It is built on top of llama.cpp, which performs the actual model inference.

What is LangChain doing in this project?

LangChain is used narrowly here — specifically its RecursiveCharacterTextSplitter for chunking. It handles the logic of trying to split text at natural boundaries (paragraphs, then sentences, then words) before falling back to raw character cuts, sparing the need to hand-write and tune that boundary-aware splitting logic from scratch.

9. Engineering Challenges Faced & How They Were Solved

Real development involves debugging, not just writing code the first time correctly. The following issues were encountered and resolved during this project — documenting them demonstrates practical troubleshooting ability, which is often as valuable to discuss in an interview as the final working code.

Library version mismatch (youtube-transcript-api)

Problem: The transcript-fetching library's API changed significantly between major versions (0.x static-method style vs. 1.x class-instance style). The installed version was explicitly checked (`pip show`) before writing code, to ensure the correct class-based syntax (`YouTubeTranscriptApi().fetch(...).to_raw_data()`) was used rather than deprecated function calls that would silently fail.

Ollama not recognized on PATH (Windows)

Problem: After installing Ollama, the command was not recognized in the terminal despite the application being correctly installed. Diagnosed by confirming the executable existed on disk (Test-Path) and could run via its full path, which isolated the issue to a stale PATH environment variable. Resolved with a full system restart, which forced Windows to refresh the environment variables system-wide.

Files created but saved empty or in the wrong location

Problem: Several new module files (e.g. `embeddings.py`, `vector_store.py`) were initially created but ended up empty or missing, causing `ModuleNotFoundError` and `AttributeError` (e.g. `'VectorStore has no len()'`) at runtime. Diagnosed by directly inspecting the file system (`dir` commands) rather than assuming the editor's save had succeeded, and fixed by fully re-pasting and re-saving complete file contents.

Partial code edits causing state drift between old and new versions

Problem: Manually editing long files in place occasionally left old code (e.g. an outdated hardcoded test-search block) coexisting with newly added code, causing confusing duplicate-logic errors. Resolved by fully replacing entire files with verified complete versions rather than attempting incremental line-level edits once drift was suspected.

Low text contrast after custom UI theming

Problem: An initial light pink theme caused several text elements (headings, captions) to blend into the background and become unreadable. Fixed by switching to a dark background with explicit, high-contrast CSS color overrides (using `!important`) targeted at Streamlit's specific internal CSS classes for headings, captions, and alert boxes.

Streamlit's rerun model losing state between interactions

Problem: Because Streamlit reruns the entire script on every UI interaction, the originally loaded video, transcript, and vector index would be lost the moment a second question was asked. Resolved by moving all persistent objects (the FAISS store, video metadata) into `st.session_state`, which survives across reruns within the same browser session.

10. Current Limitations

- **English-optimized embeddings:** all-MiniLM-L6-v2 is primarily trained on English text; semantic search quality degrades noticeably for non-English (e.g. Hindi) video transcripts.
- **No persistence:** the vector index and processed transcript are held only in memory (session_state) and are lost when the app restarts or the session ends — every video must be reprocessed from scratch each time.
- **Single-video scope:** the current system handles one video at a time; there is no cross-video comparison or multi-document retrieval.
- **No conversation memory yet:** each question is currently answered independently; natural follow-up questions (e.g. "tell me more about that") are not yet resolved using prior conversation context.
- **CPU-only inference speed:** without a dedicated GPU, LLM response generation takes several seconds per answer — acceptable for a demo, but slower than a cloud-hosted equivalent.
- **Approximate timestamps:** chunk-level timestamps are derived from the character position where a chunk begins in the concatenated transcript, so citations point to the general area of the answer rather than a frame-exact moment.
- **Not deployed publicly:** since the LLM (Ollama) must run on a local machine, the app is not currently deployed to a public cloud host in its full local-LLM form.

11. Future Improvements Roadmap

Category	Improvement	Why it matters
Retrieval quality	Hybrid search (semantic + keyword/BM25)	Catches exact names/numbers that pure semantic search can miss
Retrieval quality	Cross-encoder re-ranking of top candidates	Improves precision of the final chunks selected before generation
Retrieval quality	Multilingual embedding model	Enables reliable support for non-English video transcripts
Answer quality	Conversation memory across turns	Enables natural follow-up questions
Answer quality	Query rewriting for vague follow-ups	Improves retrieval accuracy when questions reference prior context
UX	Native chat interface (st.chat_message/st.chat_input)	More familiar, polished chatbot experience
UX	Clickable timestamps linked to an embedded player	Lets users jump directly to the cited moment in the video
UX	Auto-generated video summary and suggested questions	Gives immediate value and lowers the barrier to first use
Engineering	Persistent vector storage (disk-backed FAISS + metadata)	Avoids reprocessing the same video every session
Engineering	Centralized configuration file	Cleaner management of model names, chunk size, and other tunables
Scale	Multi-video support and cross-video comparison	Demonstrates handling of a genuinely harder, multi-document RAG problem
Delivery	Chat export and public deployment (with a hosted-LLM fallback)	Improves shareability of the finished project

12. Interview Preparation Notes

30-second explanation

"I built a YouTube RAG chatbot that lets you ask questions about any video's content and get accurate, source-cited answers — entirely using free, open-source tools running locally: Sentence Transformers for embeddings, FAISS for vector search, and Llama 3.2 via Ollama for generation, with zero API cost."

1-minute explanation

"The core problem is that YouTube videos contain valuable spoken content that's hard to search — you either watch the whole thing or scroll a messy transcript. I built a RAG pipeline that fetches a video's transcript, cleans and chunks it, embeds those chunks with a local Sentence Transformer model, and stores them in a FAISS vector index. When a user asks a question, I embed the question, retrieve the most semantically similar transcript chunks, and feed them into a locally-running Llama 3.2 model via Ollama, instructing it to answer only from that retrieved context and to say when it can't find the answer — which significantly reduces hallucination. Every component runs locally, so there's zero ongoing API cost."

3-minute technical explanation

Extend the 1-minute version with: the two-phase architecture (indexing phase vs. query phase); why `chunk_size` and overlap were chosen the way they were and that they're an empirically-tunable trade-off; why embeddings must be normalized and why that connects to using FAISS's inner-product index for cosine similarity; how the offset map preserves timestamp traceability through chunking; the separation between system prompt (constant behavior rules) and user prompt (dynamic context + question); and the deliberate hardware-aware model selection (choosing all-MiniLM-L6-v2 and Llama 3.2 3B specifically for an 8GB CPU-only machine, with documented trade-offs against larger, higher-quality alternatives).

Architecture explanation (for a whiteboard/verbal walkthrough)

Draw or describe two flows: (1) Indexing — URL → video ID → metadata → transcript → cleaning → chunking → embeddings → FAISS index (runs once per video). (2) Query — question → query embedding → FAISS search → top-K chunks → prompt construction → local LLM → grounded answer + sources (runs once per question).

Suggested resume bullet points

- Built a fully local Retrieval-Augmented Generation (RAG) chatbot that answers natural-language questions about YouTube video content, using Sentence Transformers, FAISS, and a locally-served open-source LLM (Llama 3.2 via Ollama) — with zero paid API dependency.
- Designed and implemented a modular RAG pipeline covering transcript extraction, text cleaning, timestamp-preserving chunking, semantic embedding, vector indexing, and grounded answer generation with hallucination-resistant prompting.
- Engineered the system to run entirely on consumer-grade hardware (8GB RAM, CPU-only), making deliberate, documented trade-offs in model selection (embedding model size, LLM parameter count) to balance answer quality against hardware constraints.
- Debugged and resolved real-world environment and integration issues including Python virtual environment conflicts, PATH configuration for a local LLM runtime, and Streamlit state-management across UI reruns.

Likely technical interview questions

- "Walk me through what happens from the moment a user pastes a URL to the moment they get an answer."
- "Why did you choose FAISS over a managed vector database?"
- "How do you prevent the LLM from hallucinating an answer that isn't in the video?"
- "Why normalize your embeddings, and how does that relate to your FAISS index type?"
- "What would you change if this needed to scale to millions of videos?"
- "How would you evaluate whether your retrieval is actually good?"
- "What are the current limitations of your system, and how would you address them?"

13. Conclusion

This project demonstrates a complete, working understanding of the Retrieval-Augmented Generation architecture — from raw data ingestion through to grounded, source-cited answer generation — built entirely with free, open-source, locally-run tools. Every technology choice was made deliberately, weighed against realistic alternatives, and tuned to the constraints of ordinary consumer hardware rather than assumed access to paid cloud infrastructure.

Beyond the working application itself, the process of building it — including diagnosing real environment and integration issues — reflects practical, hands-on engineering experience that goes beyond simply following a tutorial. Combined with the documented roadmap of future improvements (hybrid search, conversation memory, persistent storage, multi-video support), this project is well-positioned both as a concrete portfolio piece and as a foundation for continued, deeper development.